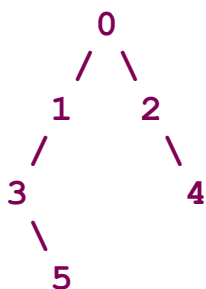


Implement depth first search algorithm and Breadth First Search algorithm, Use an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure.

```
0 -> 1, 2
1 -> 0, 3
2 -> 0, 4
3 -> 1, 5
4 -> 2
5 -> 3
```



```
package AI;

import java.util.*;

public class GraphTraversal {
    private int vertices; // Number of vertices
    private ArrayList<ArrayList<Integer>> adjList; //
Adjacency list representation

    public GraphTraversal(int vertices) {
        this.vertices = vertices;
        adjList = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    // Add an edge to the graph (undirected)
    public void addEdge(int u, int v) {
        adjList.get(u).add(v);
        adjList.get(v).add(u);
    }
}
```

```

// Recursive DFS
public void dfsRecursive(int node, boolean[] visited)
{
    visited[node] = true;
    System.out.print(node + " ");

    // Explore all neighbors
    for (int neighbor : adjList.get(node)) {
        if (!visited[neighbor]) {
            dfsRecursive(neighbor, visited);
        }
    }
}

// Helper for recursive BFS
public void bfsRecursive(Queue<Integer> queue,
boolean[] visited) {
    if (queue.isEmpty()) return;

    int node = queue.poll();
    System.out.print(node + " ");

    for (int neighbor : adjList.get(node)) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            queue.add(neighbor);
        }
    }

    bfsRecursive(queue, visited); // Recur for the
next level
}

// Main execution
public static void main(String[] args) {
    GraphTraversal graph = new GraphTraversal(6); //
Create a graph with 6 vertices (0-5)

    // Add edges
    graph.addEdge(0, 1);
    graph.addEdge(0, 2);
    graph.addEdge(1, 3);
    graph.addEdge(2, 4);
    graph.addEdge(3, 5);

```

```

// DFS
System.out.println("DFS Traversal:");
boolean[] visited = new boolean[6];
graph.dfsRecursive(0, visited);

// BFS
System.out.println("\n\nBFS Traversal:");
visited = new boolean[6];
Queue<Integer> queue = new LinkedList<>();
visited[0] = true;
queue.add(0);
graph.bfsRecursive(queue, visited);
}
}

```

dryrun

```

.....
0: [1, 2]
1: [0, 3, 4]
2: [0]
3: [1, 5]
4: [1]
5: [3]

```

- Start DFS from node = 0.
- **Visited Array** (initial state): [false, false, false, false, false, false]

Call 1: dfsRecursive(0, visited)

1. **Line 1:** visited[0] = true
 → Mark 0 as visited.
 → **Visited Array:** [true, false, false, false, false, false]
2. **Line 2:** System.out.print(0 + " ")
 → Print 0. **Output so far:** 0
3. **Line 3:** for (int neighbor : adjList.get(0))
 → adjList.get(0) = [1, 2] (neighbors of 0).
4. **Line 4:** if (!visited[neighbor])
 → First neighbor: neighbor = 1 (not visited).
 → Call dfsRecursive(1, visited).

Call 2: dfsRecursive(1, visited)

1. **Line 1:** `visited[1] = true`
 - Mark 1 as visited.
 - **Visited Array:** `[true, true, false, false, false, false]`
2. **Line 2:** `System.out.print(1 + " ")`
 - Print 1. **Output so far:** `0 1`
3. **Line 3:** `for (int neighbor : adjList.get(1))`
 - `adjList.get(1) = [0, 3, 4]` (neighbors of 1).
4. **Line 4:** `if (!visited[neighbor])`
 - First neighbor: `neighbor = 0` (already visited → skip).
 - Second neighbor: `neighbor = 3` (not visited).
 - Call `dfsRecursive(3, visited)`.

Call 3: dfsRecursive(3, visited)

1. **Line 1:** `visited[3] = true`
 - Mark 3 as visited.
 - **Visited Array:** `[true, true, false, true, false, false]`
2. **Line 2:** `System.out.print(3 + " ")`
 - Print 3. **Output so far:** `0 1 3`
3. **Line 3:** `for (int neighbor : adjList.get(3))`
 - `adjList.get(3) = [1, 5]` (neighbors of 3).
4. **Line 4:** `if (!visited[neighbor])`
 - First neighbor: `neighbor = 1` (already visited → skip).
 - Second neighbor: `neighbor = 5` (not visited).
 - Call `dfsRecursive(5, visited)`.

Call 4: dfsRecursive(5, visited)

1. **Line 1:** `visited[5] = true`
 - Mark 5 as visited.
 - **Visited Array:** `[true, true, false, true, false, true]`
2. **Line 2:** `System.out.print(5 + " ")`
 - Print 5. **Output so far:** `0 1 3 5`
3. **Line 3:** `for (int neighbor : adjList.get(5))`
 - `adjList.get(5) = [3]` (neighbors of 5).
4. **Line 4:** `if (!visited[neighbor])`
 - Neighbor: `neighbor = 3` (already visited → skip).
5. **End Call 4:** Return to **Call 3**.

Back to Call 3: Continue exploring 3's neighbors.

1. Remaining neighbors of 3 are already visited.
2. **End Call 3:** Return to **Call 2**.

Back to Call 2: Continue exploring 1's neighbors.

1. Next neighbor: `neighbor = 4` (not visited).
2. → Call `dfsRecursive(4, visited)`.

Call 5: `dfsRecursive(4, visited)`

1. **Line 1:** `visited[4] = true`
→ Mark 4 as visited.
→ **Visited Array:** `[true, true, false, true, true, true]`
2. **Line 2:** `System.out.print(4 + " ")`
→ Print 4. **Output so far:** `0 1 3 5 4`
3. **Line 3:** `for (int neighbor : adjList.get(4))`
→ `adjList.get(4) = [1]` (neighbors of 4).
4. **Line 4:** `if (!visited[neighbor])`
 - Neighbor: `neighbor = 1` (already visited → skip).
5. **End Call 5:** Return to **Call 2**.

Back to Call 2: Continue exploring 1's neighbors.

1. Remaining neighbors of 1 are already visited.
2. **End Call 2:** Return to **Call 1**.

Back to Call 1: Continue exploring 0's neighbors.

1. Next neighbor: `neighbor = 2` (not visited).
2. → Call `dfsRecursive(2, visited)`.

Call 6: `dfsRecursive(2, visited)`

1. **Line 1:** `visited[2] = true`
→ Mark 2 as visited.
→ **Visited Array:** `[true, true, true, true, true, true]`
2. **Line 2:** `System.out.print(2 + " ")`
→ Print 2. **Output so far:** `0 1 3 5 4 2`
3. **Line 3:** `for (int neighbor : adjList.get(2))`
→ `adjList.get(2) = [0]` (neighbors of 2).

4. **Line 4:** `if (!visited[neighbor])`
 - Neighbor: `neighbor = 0` (already visited → skip).
5. **End Call 6:** Return to **Call 1**.

Back to Call 1: Continue exploring 0's neighbors.

1. All neighbors of 0 are visited.
2. **End Call 1:** DFS complete.

Final Output

The DFS traversal prints the vertices in this order:

0 1 3 5 4 2

```
.....
// BFS

System.out.println("\n\nBFS Traversal:");
visited = new boolean[6];
Queue<Integer> queue = new LinkedList<>();
visited[0] = true;
queue.add(0);
graph.bfsRecursive(queue, visited);
.....
```

Initial Setup:

- Start at vertex 0.
- Initialize the visited array: `visited = [false, false, false, false, false, false]`.
- Create an empty queue: `queue = []`.
- Add the starting node 0 to the queue: `queue = [0]`.
- Mark node 0 as visited: `visited = [true, false, false, false, false, false]`.

First Call to `bfsRecursive(queue, visited)`:

1. **Queue:** `[0]`
 - **Node:** 0 (dequeue node 0)
 - Print 0.
 - **Neighbors of 0:** 1, 2
 - **Visit neighbor 1:** `visited = [true, true, false, false, false, false]`, `queue = [1]`.
 - **Visit neighbor 2:** `visited = [true, true, true, false, false, false]`, `queue = [1, 2]`.

Second Call to bfsRecursive(queue, visited):

2. Queue: [1, 2]

- **Node: 1** (dequeue node 1)
- Print 1.
- **Neighbors of 1: 0, 3**
- **Neighbor 0 is already visited**, so we skip it.
- **Visit neighbor 3:** visited = [true, true, true, true, false, false], queue = [2, 3].

Third Call to bfsRecursive(queue, visited):

3. Queue: [2, 3]

- **Node: 2** (dequeue node 2)
- Print 2.
- **Neighbors of 2: 0, 4**
- **Neighbor 0 is already visited**, so we skip it.
- **Visit neighbor 4:** visited = [true, true, true, true, true, false], queue = [3, 4].

Fourth Call to bfsRecursive(queue, visited):

4. Queue: [3, 4]

- **Node: 3** (dequeue node 3)
- Print 3.
- **Neighbors of 3: 1, 5**
- **Neighbor 1 is already visited**, so we skip it.
- **Visit neighbor 5:** visited = [true, true, true, true, true, true], queue = [4, 5].

Fifth Call to bfsRecursive(queue, visited):

5. Queue: [4, 5]

- **Node: 4** (dequeue node 4)
- Print 4.
- **Neighbors of 4: 2**
- **Neighbor 2 is already visited**, so we skip it.

Sixth Call to bfsRecursive(queue, visited):

6. Queue: [5]

- **Node: 5** (dequeue node 5)
- Print 5.
- **Neighbors of 5: 3**
- **Neighbor 3 is already visited**, so we skip it.

visited = [true, true, true, true, true, true]

queue = []

Recur with queue = [].

Step 7: Function Call with queue = []

1. **Check if queue is empty:**

- queue is empty, so return and terminate recursion.

Output:

The BFS traversal prints:

BFS Traversal:

0 1 2 3 4 5

.....